Individual Assignment 2 - Tron Snake Arena

ENG1010 - Algorithms and Data Structures

1 Introduction

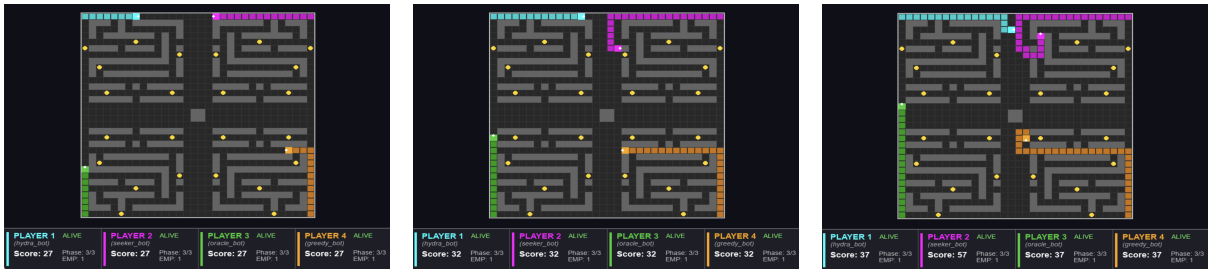

Figure 1: Gameplay Overview Sequence

This game is a grid-based, 4-player strategy game. Each player controls a snake that leaves a solid trail behind it. The map consists of obstacles, power-ups, and points. Your goal is to design an algorithm (a "Bot") that navigates the grid to achieve the highest possible score.

2 Game Behaviour

The game operates on a simultaneous movement system. At every tick, all 4 players move at the same time. Your bot will receive full information about the current state of the game: grid layout, player positions, and other metadata. Specifically, you must implement a `Bot` class containing the following method:

```
class Bot:
    def get_move(self, board, player_id, pos, info):
        # Your logic here
        return move_command
```

Here, `board` is a 2D $N \times M$ grid representing the map, `player_id` is your integer ID, `pos` is your (x, y) coordinate, and `info` is a dictionary containing extra state data (scores, cooldowns, map name, etc.).

3 Available Moves

At each turn, your player must make a decision. Your `get_move` function MUST return exactly one of the following strings.

Moving in 4 Directions By returning the strings `'N'`, `'S'`, `'W'`, or `'E'`, you instruct your snake to move North, South, West, or East, respectively. Note that it is possible to instruct your snake to move "inside" your own body (backwards), but this will result in an immediate collision and game over. Be careful!

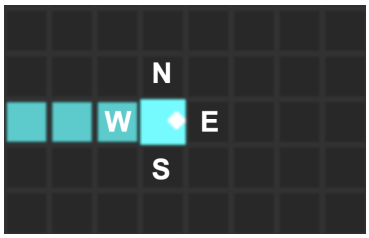

Figure 2: 4-Directional Movement

Boost By returning the string `'+'`, you instruct your snake to boost. When boosting, you move **2 spaces** in the current direction instantly. This is useful for catching points quickly or blocking/trapping opponents.

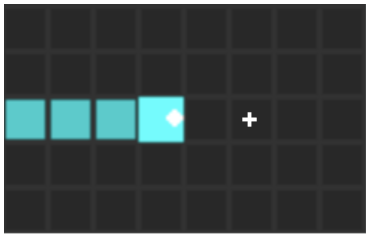

Figure 3: Boost Mechanic. When boosting, player can move 2 spaces in the current direction.

Phase By returning the string `'P'`, you instruct your snake to Phase. Similar to boost, you will move 2 spaces in the current direction. However, unlike boost, phasing allows you to ignore collisions with any object *between* your start and end point (though not the specific space you land on). This is useful for skipping over barriers or escaping enemies. Phase has a limit of **3 uses per game**.

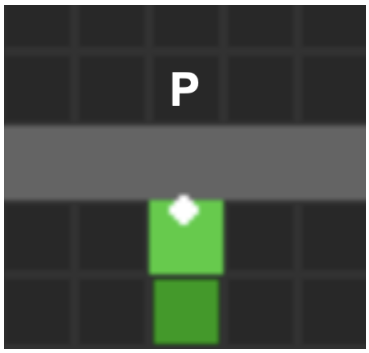

Figure 4: Phase Mechanic. When phasing, player can avoid hitting obstacles.

EMP-Blast By returning the string `'X'`, you instruct your snake to use an EMP blast. Upon activation, a blue energy field (5×5 area) will appear around your snake's head. After **5 ticks**, the blast launches! Any player caught in the blast radius will be stunned for **7 turns**.

Note that EMP-Blast is different from other moves. You will also need to return other moves alongside the EMP-Blast as the next string. For example, returning `'XN'` will trigger EMP-Blast and move your bot north. Similarly, returning `'X+'` will trigger EMP-Blast and boost.

Figure 5: EMP-Blast. The Green player just initiate the EMP a turn ago. Blue highlights the area-of-effect, which will follow the Green player's head. 4 turns later, the EMP hits the Purple player, stunning it (with X mark on the head). Purple player cannot move and eventually will hit the Green player's trail.

Time Limit Your bot must return a move within 0.5s every step. Taking more than 0.5s will lead to a warning. 5 warnings and your bot will be automatically destroyed. However, if you take more than 2 seconds, you will also be automatically destroyed without warning. Any error will also automatically destroy your bot.

4 Game Map

The game is played on a 2D grid of size $N \times M$ (guaranteed not to exceed 50×50). The `board` parameter captures the current map elements.

Empty Area This is a walkable area, denoted as `'.'` in the board.

Wall This is a standard obstacle. Hitting it destroys your character, though you can Phase through it. It is denoted as `'#'`.

Timed Wall Similar to a wall, hitting it destroys your character (unless Phasing). However, this wall will collapse into empty space after a set time. It is denoted as an **integer**, representing the turns remaining until it opens.

Player Trail Each player leaves a trail that acts like a wall. Hitting it destroys your character. It is denoted as `'t1'` to `'t4'`, corresponding to the player ID.

Coins and Diamonds Collecting these items grants points. A Coin is denoted as `'c'` (+20 pts) and a Diamond as `'D'` (+50 pts).

5 Point System

The winner is decided by the highest score.

- **Each Turn:** +1 point for every turn survived, or +2 if using boost or phase.
- **Survival:** The last player to survive will gain an additional +50 points, the 2nd last player to survive ill gain +25, and the 3rd one to survive +10 points.
- **Coin ('c'):** +20 points.
- **Diamond ('D'):** +50 points.
- **EMP Hit:** +50 points per opponent stunned.

6 Assignment Grade

You will receive the following placement points depending on your final position in a match:

- **1st position:** 100 points
- **2nd position:** 50 points
- **3rd position:** 25 points
- **4th position:** 0 points

Note: In the Duel scenario, placement points are strictly win/loss: a win (1st position) awards 100 points, and a loss (2nd position) awards 0 points.

Your algorithm will be evaluated based on its overall performance against various opponent bots and map designs.

10 Points: Solo Player You will be tested on the following maps. No other bots will intervene, and your goal is to achieve the highest possible score. In this setting, you will be Player 1, and Players 2-4 will be *Drunk* bots.

Maps:

- `s_path_0`, `s_path_1`, `s_path_2`, `s_path_3`
- `s_floodfill_0`, `s_floodfill_1`, `s_floodfill_2`
- `s_choice_0`, `s_choice_1`, `s_choice_2`

30 Points: Duel You will be tested in a 1v1 setting against another bot. In this setting, both competing players will start as Player 1 and Player 4, while Player 2 and Player 3 will be *Dummy* bots (which do nothing). Each map will be played twice, allowing each player to try both starting positions.

You will be tested against the combinations of all maps and enemy bots as follows:

Maps: `arena`, `maze`, `treasure`, `gate`, `cube`, `orbit`
Bots: *Scout*, *Rogue*, *Stingray*, *Viper*, *Blaze*

15 Points: Battle Royale You will be tested in an all-vs-all format across the following settings:

Map	Bot 1	Bot 2	Bot 3	Bot 4
`arena`	You	Scout	Stingray	Blaze
`arena`	You	Rogue	Viper	Stingray
`arena`	You	Rogue	Viper	Blaze
`treasure`	You	Rogue	Stingray	Stingray
`treasure`	You	Viper	Viper	Rogue
`treasure`	You	Rogue	Scout	Blaze
`maze`	You	Rogue	Rogue	Scout
`maze`	You	Viper	Rogue	Rogue
`gate`	You	Stingray	Stingray	Blaze
`gate`	You	Viper	Stingray	Scout
`cube`	You	Stingray	Stingray	Blaze
`cube`	You	Viper	Stingray	Scout
`cube`	You	Viper	Blaze	Blaze
`orbit`	You	Rogue	Rogue	Viper
`orbit`	You	Blaze	Viper	Scout

You can run the evaluation code to automatically evaluate your bot with the public test. See Section 8.4

25 Points: Hidden Scenario You will be tested in more solo player (8 points), and against a new bot and/or a new map in either a Duel (10 points) or Battle Royale (7 points) scenario. For the hidden solo player, it is guaranteed that the map will be a variation of path, floodfill, or choice. The name will also follow the same format, e.g., `s_path_N`, so you can use this as your advantage, as you will have access to the map name.

10 Points: Student vs Student You will be tested against all other students on a random map.

20 Points: Report See in the report section. Although this is only 20 points, any **submission without a sufficient report will be penalized.**

For the report, we are looking for 3 parts:

- **Algorithm**: Precisely explain your algorithm
- **Complexity Analysis**: Analyze your complexity, both space and time.
- **Observation**: Observe your algorithm, explain cases where your bot works well, and explain cases where your bot is subotimal. Explain why.

Note: The maximum possible score for this assignment is 110 points; therefore, any grade exceeding 100 will be capped at 100.

7 How To Run The Game

First, pull the code from `https://github.com/afaji/tron_snake_ENG1010_2026/tree/main`. You will need a Python3.11 and all required packages in the requirement file.

7.1 Prerequisites

Before starting, ensure you have the following installed:

- **Python 3.11** (Required version for compatibility)
- **Git** (Configured with SSH keys for repository access)

8 Environment Setup

8.1 Cloning the Repository

Clone the project to your local machine:

```
git clone git@github.com:afaji/tron_snake_ENG1010_2026.git
cd tron_snake_ENG1010_2026
```

8.2 Virtual Environment

It is best practice to use a virtual environment to manage dependencies:

```
# Create the environment
python3.11 -m venv .venv

# Activate (macOS/Linux)
source .venv/bin/activate

# Activate (Windows)
# .venv\Scripts\activate
```

8.3 Installing Dependencies

Install the required packages using `pip`:

```
pip install --upgrade pip
pip install -r requirements.txt
```

8.4 Execution and Evaluation

To run and visualize the game with GUI, run the following:

```
python main.py
```

For faster simulations with CLI-based interaction:

```
python runner.py -h
```

To verify your implementation against the public testcases:

```
chmod +x evaluate.sh
./evaluate.sh MyCoolBot
```

Note that MyCoolBot is your bot name. You should name your file correctly. Implement your bot and put it in bots folder. Feel free to use any cool bot name.

9 Submission (deadline April 13, 10am Abu Dhabi Time)

Your code and your report will be submitted using different submission links that will be provided in the LMS. Also, include in your report the **declaration of original work**, as in the previous assignment.

9.1 Submission: Bots

You need to submit your bot python code. ONLY submit the python file and nothing else, eg `MyCoolBot.py`. DO NOT ZIP, Compress, or submit anything else but a python source code. The grading will be done automatically and failure to follow this instruction might affect your grade.

9.2 Submission: Report

You are required to document the algorithm implemented in your bot. Detail your overall approach, the time and space complexity, and specific scenarios where your bot performs well or poorly.

10 Known Features

The following are not bugs, but a feature.

- Selector issue: You can press keys to search for maps or bots, but sometimes it does not register. You will need to actually click the option to register.
- When contesting for points, if 2 or more bots are trying to get the same coin or diamond, those with smaller ID will get it as priority. Use this as part of your strategy.
- Should other bugs be discovered, the game engine will stay as it is, unless the new bug is groundbreaking ones, in which case we will let the students know. Hence, feel free to utilize the bug/glitches in terms of game logic as part of the strategy.

11 Bonus Task: Weekly Puzzles (Optional)

As an optional bonus component, students may submit solutions to the **last 10 weekly puzzles**.

- Each correctly completed puzzle is worth **1% bonus**, for a maximum of **10%** of this assignment.
- Only the **last 10 puzzles** released during the semester are eligible for this bonus.
- Submissions must be complete and demonstrate clear reasoning to receive credit.

Bonus Policy:

- Bonus marks will be applied to **one of the following components only**:
  - Graded Quiz
  - Individual Assignment 1
  - Individual Assignment 2
- The bonus **will not be applied** to the **Midterm** or **Final Exam**.
- The application of bonus marks will be capped in such a way that no component exceeds its maximum possible grade.

Note: The bonus marks are intended to reward consistent participation and problem-solving practice. They are not a substitute for core assessment performance.

12 Grading Rubric

Note: The implementation (bot performance) is automatically graded using the evaluation framework described in Section 6 of the assignment brief. Students should focus on both achieving strong performance and clearly documenting their approach in this report.

Criterion	Marks	Poor	Average	Good	Very Good	Excellent
Algorithm Explanation	8	Missing or unclear description of the approach	Basic description; lacks clarity or detail	Clear explanation of main idea and logic	Well-structured explanation with justification of decisions	Precise, insightful explanation with strong reasoning and clarity
Complexity Analysis	6	Missing or incorrect analysis	Complexity stated without justification	Correct Big-O time/space complexity stated	Correct analysis with reasonable explanation	Thorough, well-justified analysis with comparisons and discussion
Observations & Evaluation	6	Missing or incorrect discussion	Superficial observations with little insight	Reasonable interpretation of results and behaviour	Thoughtful discussion linked to performance and scenarios	Deep insights explaining strengths, weaknesses, and algorithm behaviour

Table 1: Report Rubric for Tron Snake Arena Assignment. Performance and code evaluation are assessed separately based on the point system described in Section 6 of the assignment brief.